SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week6 – Monday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Name	S.Hasitha varshini	**Hallticket**	2303A54038
Assignment Number: 11.1(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*
1	**Lab 11 – Data Structures with AI: Implementing Fundamental Structures** **Lab Objectives** • Use AI to assist in designing and implementing fundamental data structures in Python. • Learn how to prompt AI for structure creation, optimization, and documentation. • Improve understanding of Lists, Stacks, Queues, Linked Lists, Trees, Graphs, and Hash Tables. • Enhance code quality with AI-generated comments and performance suggestions. --- **Task Description #1 – Stack Implementation** Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code: class Stack: pass Expected Output: • A functional stack implementation with all required methods and docstrings.	Week6 -Monday

```python
# Task-1
# Use AI to generate a Stack class with push, pop, peek, and is_empty methods. Sample Input Code:class Stack:passExpected Output:• A functional stack implementation with all required methods anddocstrings.
class Stack:
    """
    A simple implementation of a stack data structure.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises IndexError if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises IndexError if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.items) == 0
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    assert stack.peek() == 3
    assert stack.pop() == 3
    assert stack.peek() == 2
    assert not stack.is_empty()
    stack.pop()
    stack.pop()
    assert stack.is_empty()
    try:
        stack.pop()
    except IndexError as e:
        assert str(e) == "Pop from an empty stack"
    try:
        stack.peek()
    except IndexError as e:
        assert str(e) == "Peek from an empty stack"
    print("All tests passed!")
```

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
All tests passed!
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
All tests passed!
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

  pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

```python
#TASK2
from collections import deque

class Queue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, item):
        self.queue.append(item)
        print(item, "added to queue")

    def dequeue(self):
        if len(self.queue) == 0:
            print("Queue is empty")
            return None
        return self.queue.popleft()

    def peek(self):
        if len(self.queue) == 0:
            print("Queue is empty")
            return None
        return self.queue[0]

    def size(self):
        return len(self.queue)


q = Queue()
q.enqueue(5)
q.enqueue(15)
q.enqueue(25)

print("Front element:", q.peek())
print("Dequeued:", q.dequeue())
print("Queue size:", q.size())
```

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1 …
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.p
All tests passed!
5 added to queue
15 added to queue
25 added to queue
Front element: 5
Dequeued: 5
Queue size: 2
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display

methods.

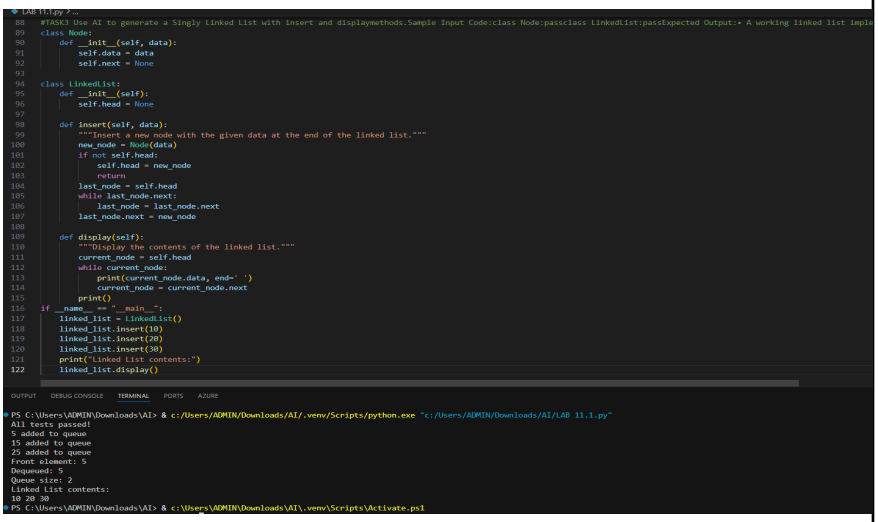
Sample Input Code:

```
class Node:
    pass

class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.



```
LAB 11.1.py > ...
#TASK3 Use AI to generate a Singly Linked List with insert and displaymethods.Sample Input Code:class Node:passclass LinkedList:passExpected Output:• A working linked list implem
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the linked list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List contents:")
    linked_list.display()
```

```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
All tests passed!
5 added to queue
15 added to queue
25 added to queue
Front element: 5
Dequeued: 5
Queue size: 2
Linked List contents:
10 20 30
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

```
class BST:
    pass
```

Expected Output:

- BST implementation with recursive insert and traversal methods.

```python
#Task4Task Description #4 – Binary Search Tree (BST)Task: Use AI to create a BST with insert and in-order traversal methods.Sample Input Code:class BSTpassExpected Output:• BST implementation with recursive insert and traversal methods
class Node:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key

class BST:
    def __init__(self):
        self.root = None

    def insert(self, key):
        """Insert a new node with the given key into the BST."""
        if self.root is None:
            self.root = Node(key)
        else:
            self._insert_recursive(self.root, key)

    def _insert_recursive(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = Node(key)
            else:
                self._insert_recursive(node.left, key)
        else:
            if node.right is None:
                node.right = Node(key)
            else:
                self._insert_recursive(node.right, key)

    def in_order_traversal(self):
        """Perform in-order traversal of the BST and return a list of values."""
        return self._in_order_recursive(self.root)

    def _in_order_recursive(self, node):
        res = []
        if node:
            res = self._in_order_recursive(node.left)
            res.append(node.val)
            res = res + self._in_order_recursive(node.right)
        return res
if __name__ == "__main__":
    bst = BST()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)

    print("In-order Traversal of BST:", bst.in_order_traversal())
```

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
In-order Traversal of BST: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/Activate.ps1
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

  pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```python
#Task5: Use AI to implement a hash table with basic insert, search, anddelete methods.Sample Input Code:class HashTable:passExpected Output:• Collision handling using chaining, with well-commented methods.
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate a hash for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))

    def search(self, key):
        """Search for a value by its key. Returns the value or None if not found."""
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False
if __name__ == "__main__":
    hash_table = HashTable()
    hash_table.insert("name", "Alice")
    hash_table.insert("age", 30)
    hash_table.insert("city", "New York")

    print("Search for 'name':", hash_table.search("name"))
    print("Search for 'age':", hash_table.search("age"))
    print("Search for 'city':", hash_table.search("city"))

    print("Deleting 'age'...")
    hash_table.delete("age")
    print("Search for 'age' after deletion:", hash_table.search("age"))
```

```
OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py" …
(.venv) PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
Search for 'name': Alice
Search for 'age': 30
Search for 'city': New York
Deleting 'age'...
Search for 'age' after deletion: None
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```
LAB 11.1.py > ...
#TASK6:Use AI to implement a graph using an adjacency list.Sample Input Code:class Graph:passExpected Output:• Graph with methods to add vertices, add edges, and disp
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph."""
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)  # For undirected graph

    def display(self):
        """Display the graph as an adjacency list."""
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {', '.join(edges)}")
if __name__ == "__main__":
    graph = Graph()
    graph.add_vertex("A")
    graph.add_vertex("B")
    graph.add_vertex("C")
    graph.add_edge("A", "B")
    graph.add_edge("A", "C")
    graph.add_edge("B", "C")

    print("Graph connections:")
    graph.display()

OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE                                   Python

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
Graph connections:
A: B, C
B: A, C
C: A, B
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```
# Task-7
# Use AI to implement a priority queue using Python's heapq module.Expected Output:• Implementation with enqueue (priority), dequeue (highest priority),and display metho
import heapq

class PriorityQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, priority, item):
        heapq.heappush(self.queue, (priority, item))

    def dequeue(self):
        if not self.queue:
            raise IndexError("dequeue from an empty priority queue")
        return heapq.heappop(self.queue)[1]

    def display(self):
        for priority, item in self.queue:
            print(f"Priority: {priority}, Item: {item}")
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue(2, "Task 2")
    pq.enqueue(1, "Task 1")
    pq.enqueue(3, "Task 3")

    print("Priority Queue contents:")
    pq.display()

    print("\nDequeueing items:")
    while True:
        try:
            print(pq.dequeue())
        except IndexError:
            print("Priority Queue is empty")
            break

OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
Priority Queue contents:
Priority: 1, Item: Task 1
Priority: 2, Item: Task 2
Priority: 3, Item: Task 3

Dequeueing items:
Task 1
Task 2
Task 3
Priority Queue is empty
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

  pass

Expected Output:

- Insert and remove from both ends with docstrings.

LAB 11.1.py

```python
# Task-8
# Use AI to implement a double-ended queue using collections.deque.Expected Output:• Insert and remove from both ends with docstrings.
from collections import deque
class Deque:
    def __init__(self):
        """Initialize an empty double-ended queue."""
        self.deque = deque()

    def add_front(self, item):
        """Add an item to the front of the deque."""
        self.deque.appendleft(item)

    def add_rear(self, item):
        """Add an item to the rear of the deque."""
        self.deque.append(item)

    def remove_front(self):
        """Remove and return an item from the front of the deque. Raises IndexError if the deque is empty."""
        if not self.deque:
            raise IndexError("remove from an empty deque")
        return self.deque.popleft()

    def remove_rear(self):
        """Remove and return an item from the rear of the deque. Raises IndexError if the deque is empty."""
        if not self.deque:
            raise IndexError("remove from an empty deque")
        return self.deque.pop()

    def display(self):
        """Display the contents of the deque."""
        print("Deque contents:", list(self.deque))
if __name__ == "__main__":
    d = Deque()
    d.add_rear("Task 1")
    d.add_rear("Task 2")
    d.add_front("Task 0")
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE

```
PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
After adding tasks:
Deque contents: ['Task 0', 'Task 1', 'Task 2']

Removing from front: Task 0
Removing from rear: Task 2

After removals:
Deque contents: ['Task 1']
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
    - Stack
    - Queue
    - Priority Queue
    - Linked List
    - Binary Search Tree (BST)
    - Graph
    - Hash Table
    - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

LAB 11.1.py

```
LAB 11.1.py > ...

#TASK3Generate a solution for a Campus Resource Management System by selecting suitable data structures (Stack, Queue, Priority Queue, Linked List, BST, Graph, Hash Table, Deque) for features like attendance
# Campus Resource Management System - Data Structure Selection
# Feature                  | Data Structure        | Justification
#-------------------------|----------------------|----------------------
# Attendance Tracking      | Hash Table            | Fast lookups for student attendance records.
# Event Registration       | Queue                 | First-come, first-served basis for event registration.
# Library Borrowing        | Linked List           | Dynamic list of borrowed books with easy insertion and deletion.
# Bus Scheduling           | Graph                 | Represents bus routes and stops effectively.
# Cafeteria Order Management | Priority Queue      | Prioritize orders based on preparation time or
# customer type (e.g., staff vs. student).

# Implementing the Cafeteria Order Management feature using a Priority Queue
import heapq
class CafeteriaOrderManagement:
    def __init__(self):
        """Initialize an empty priority queue for orders."""
        self.orders = []

    def place_order(self, order_id, priority):
        """Place a new order with a given priority."""
        heapq.heappush(self.orders, (priority, order_id))
        print(f"Order {order_id} placed with priority {priority}.")

    def serve_order(self):
        """Serve the highest priority order. Raises IndexError if no orders are present."""
        if not self.orders:
            raise IndexError("No orders to serve")
        priority, order_id = heapq.heappop(self.orders)
        print(f"Serving order {order_id} with priority {priority}.")
        return order_id

    def display_orders(self):
        """Display all pending orders sorted by priority."""
        print("Pending Orders:")
        for priority, order_id in sorted(self.orders):
            print(f"Order ID: {order_id}, Priority: {priority}")
if __name__ == "__main__":
    cafeteria = CafeteriaOrderManagement()
    cafeteria.place_order("Order1", 2)
    cafeteria.place_order("Order2", 1)
    cafeteria.place_order("Order3", 3)

    cafeteria.display_orders()

    cafeteria.serve_order()
    cafeteria.serve_order()

    print("\nAfter serving some orders:")
    cafeteria.display_orders()
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE Python

```
PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
Order Order1 placed with priority 2.
Order Order2 placed with priority 1.
Order Order3 placed with priority 3.
Pending Orders:
Order ID: Order2, Priority: 1
Order ID: Order1, Priority: 2
Order ID: Order3, Priority: 3
Serving order Order2 with priority 1.
Serving order Order1 with priority 2.

After serving some orders:
Pending Orders:
Order ID: Order3, Priority: 3
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\ADMIN\Downloads\AI>
```

Task Description #10: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```python
# Prompt: Generate a solution for a Smart E-Commerce Platform by selecting appropriate data structures (Stack, Queue, Priority Queue, Linked List, Binary Search Tree, Graph, Hash Table, Deque) for features such as Shopping Cart Management, Order Processing, Top-Selling Products Tracker, Product Search Engine, Delivery Route Planning.
# Smart E-Commerce Platform - Data Structure Selection
# Feature                  | Data Structure      | Justification
# -------------------------|---------------------|-------------------------------------------------------------
# Shopping Cart Management | Linked List         | Allows dynamic addition and removal of items in the cart with efficient memory usage.
# Order Processing         | Queue               | Ensures orders are processed in the order they were received, following a first-come, first-served approach.
# Top-Selling Products Tracker | Hash Table      | Provides fast access and updates to product sales data for tracking top-selling items.
# Product Search Engine    | Binary Search Tree  | Enables efficient searching, insertion, and deletion of products based on attributes like name or price.
# Delivery Route Planning  | Graph               | Represents delivery routes and locations effectively, allowing for pathfinding algorithms to optimize routes.
# Implementing the Product Search Engine feature using a Binary Search Tree
class ProductNode:
    def __init__(self, product_id, name, price):
        """Initialize a product node with ID, name, and price."""
        self.product_id = product_id
        self.name = name
        self.price = price
        self.left = None
        self.right = None
class ProductSearchEngine:
    def __init__(self):
        """Initialize an empty binary search tree for products."""
        self.root = None

    def insert_product(self, product_id, name, price):
        """Insert a new product into the BST based on product ID."""
        if self.root is None:
            self.root = ProductNode(product_id, name, price)
        else:
            self._insert_recursive(self.root, product_id, name, price)

    def _insert_recursive(self, node, product_id, name, price):
        if product_id < node.product_id:
            if node.left is None:
                node.left = ProductNode(product_id, name, price)
            else:
                self._insert_recursive(node.left, product_id, name, price)
        else:
            if node.right is None:
                node.right = ProductNode(product_id, name, price)
            else:
                self._insert_recursive(node.right, product_id, name, price)

    def search_product(self, product_id):
        """Search for a product by its ID. Returns the product details or None if not found."""
        return self._search_recursive(self.root, product_id)

    def _search_recursive(self, node, product_id):
        if node is None:
            return None
        if product_id == node.product_id:
            return (node.name, node.price)
        elif product_id < node.product_id:
            return self._search_recursive(node.left, product_id)
        else:
            return self._search_recursive(node.right, product_id)
if __name__ == "__main__":
    search_engine = ProductSearchEngine()
    search_engine.insert_product(101, "Laptop", 999.00)
    search_engine.insert_product(102, "Smartphone", 499.99)
    search_engine.insert_product(103, "Headphones", 149.99)

    product = search_engine.search_product(102)
    if product:
        print(f"Product found: Name: {product[0]}, Price: $${product[1]}")
    else:
        print("Product not found.")
```

C

```
OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    AZURE                                   Python
PS C:\Users\ADMIN\Downloads\AI> & c:/Users/ADMIN/Downloads/AI/.venv/Scripts/python.exe "c:/Users/ADMIN/Downloads/AI/LAB 11.1.py"
Product found: Name: Smartphone, Price: $499.99
PS C:\Users\ADMIN\Downloads\AI> & c:\Users\ADMIN\Downloads\AI\.venv\Scripts\Activate.ps1
```